

DQN

LAMDA-5

April 9, 2026

1 DQN 算法设计

DQN (Deep Q-Network, 深度 Q 网络) 是一种将深度学习与强化学习中的 Q 学习相结合的算法。它在 2013 年由 DeepMind 提出, 并在 2015 年进行了完善, 因在 Atari 游戏上超越人类水平的表现而闻名。

在 DQN 出现之前, 传统的 Q 学习算法主要依赖于 Q table 来存储状态-动作的价值。这种方法在处理高维度、连续的状态空间 (如游戏画面、机器人视觉输入) 时会遇到根本性困难:

- **维度灾难**: 如果直接用表格存储, 状态空间会巨大无比 (例如, 一个 Atari 游戏的画面像素组合几乎是无限的), 导致 Q 表无法构建和更新。
- **泛化能力差**: 表格方法无法将学到的知识从见过的状态推广到未见过的相似状态。

DQN 将 Q 函数用神经网络 $Q(s, a; \theta)$ 来近似, 通过最小化如下损失函数来训练:

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(y - Q(s, a; \theta))^2 \right], \quad y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

为了解决训练不稳定的问题, DQN 创新性地引入了两个重要机制:

- **经验回放 (Replay Buffer)**: 强化学习采集的样本 (s_t, a_t, r_t, s_{t+1}) 通常是序列相关的, 如果直接按顺序学习, 会导致模型训练不稳定。DQN 引入 Replay Buffer, 将智能体与环境互动的样本存储起来, 训练时从中**随机采样**。这打破了数据之间的相关性, 使得数据更接近独立同分布 (i.i.d.), 提高了学习的稳定性。此外, 每条经验可以被多次复用, 提升了样本效率。
- **目标网络 (Target Network)**: 在 Q-learning 中, 如果用同一个网络计算当前 Q 值和目标 Q 值, 两者会相互影响, 造成 "Moving Target" 问题, 容易发散。DQN 使用两个结构相同但参数更新频率不同的网络——评估网络 θ 和目标网络 θ^- 。目标网络的参数每隔固定步数才从评估网络复制 (硬更新), 或者以极小步长缓慢跟踪 (软更新)。这暂时固定了优化目标, 显著提升了算法的收敛稳定性。

算法的完整伪代码如下 (来自原论文):

Algorithm 1 Deep Q-learning with Experience Replay

```
1: Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequenced  $\phi_1 = \phi(s_1)$ 
6:   for  $t = 1, T$  do
7:     With probability  $\epsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
9:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
10:    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
12:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
13:    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
14:    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$ 
15:  end for
16: end for
```

2 Playing Atari with Deep Reinforcement Learning

仔细阅读文章“Playing Atari with Deep Reinforcement Learning” (Mnih et al., DeepMind, 2013), 以下是对论文核心内容的梳理, 重点关注实验部分。

2.1 问题设定

论文将 Atari 游戏建模为 MDP: 智能体在每个时间步观察原始像素图像 $x_t \in \mathbb{R}^d$, 选择动作 $a_t \in \mathcal{A}$, 获得标量奖励 r_t 。由于单帧画面存在遮挡 (部分可观测), 将最近 4 帧叠加为状态 $s_t = \{x_{t-3}, x_{t-2}, x_{t-1}, x_t\}$ 。目标是最大化折扣累积回报 $R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ 。最优动作值函数满足 Bellman 方程:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

2.2 网络架构

输入为 $84 \times 84 \times 4$ 的预处理帧 (原始 210×160 RGB 先灰度化再裁剪缩放)。网络结构如下:

层	类型	滤波器/单元	卷积核/—	步长
输入	—	—	$84 \times 84 \times 4$	—
Conv1	卷积 +ReLU	16	8×8	4
Conv2	卷积 +ReLU	32	4×4	2
FC1	全连接 +ReLU	256	—	—
输出	全连接 (线性)	$ \mathcal{A} $	—	—

输出层为每个合法动作输出一个 Q 值, 单次前向传播即可计算所有动作的 Q 值, 效率远高于“(s,a)→Q”的设计。

2.3 训练细节

- 优化器: RMSProp, minibatch 大小 32
- ϵ -greedy: ϵ 从 1 线性退火至 0.1 (前 100 万帧), 之后固定 0.1
- 总训练帧数: 1000 万帧
- Replay Memory: 存储最近 100 万帧的转移样本
- 奖励裁剪: 所有正奖励设为 +1, 负奖励设为 -1, 使不同游戏可以共用同一学习率
- 帧跳过 (frame skip): 每 $k = 4$ 帧选择一次动作 (Space Invaders 用 $k = 3$ 避免激光不可见)

2.4 论文实验结果

论文在 7 款 Atari 游戏上评测, 与以往最优方法对比如下:

方法	B.Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S.Invaders
Random	354	1.2	0	-20.4	157	110	179
Sarsa	996	5.2	129	-19	614	665	271
Contingency	1743	6	159	-17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	-3	18900	28010	3690

DQN 在全部 7 款游戏上以较大优势超越所有已有学习方法, 并在 Breakout、Enduro、Pong 三款游戏上超越人类专家。其在 Pong 上的均值为 20 (满分 21), 与本次实验目标一致。论文还发现, 平均 Q 值曲线比奖励曲线更平滑稳定, 是衡量训练进展的良好指标。

3 TODO: 代码实现

具体实现步骤参考代码中的 README, 主要步骤为配置环境、完成代码中 TODO 部分。以下展示各模块的核心实现。

3.1 utils/networks.py: CNN 网络架构

按照论文描述，实现两层卷积加一层全连接的网络。空间尺寸推导：Conv1 $(84 - 8)/4 + 1 = 20$ ，Conv2 $(20 - 4)/2 + 1 = 9$ ，Flatten: $32 \times 9 \times 9 = 2592$ 。

```
1 class AtariDQNNetwork(nn.Module):
2     def __init__(self, input_shape, num_actions):
3         super(AtariDQNNetwork, self).__init__()
4         self.conv1 = nn.Conv2d(4, 16, 8, stride=4) # (B,4,84,84) -> (B,16,20,20)
5         self.conv2 = nn.Conv2d(16, 32, 4, stride=2) # (B,16,20,20) -> (B,32,9,9)
6         self.fc1 = nn.Linear(32 * 9 * 9, 256) # 2592 -> 256
7         self.fc2 = nn.Linear(256, num_actions) # 256 -> |A|
8
9     def forward(self, x):
10        x = F.relu(self.conv1(x)) # 提取低级特征 (边缘、纹理)
11        x = F.relu(self.conv2(x)) # 提取高级特征
12        x = x.view(x.size(0), -1) # 展平: (B, 2592)
13        x = F.relu(self.fc1(x)) # 全连接
14        return self.fc2(x) # 输出各动作Q值 (无激活函数, Q可为负)
```

Listing 1: AtariDQNNetwork 网络定义

3.2 agent/atari_agent.py: Q 值计算与动作选择

3.2.1 select_action

实现确定性策略 (argmax) 和 softmax 探索策略:

$$\pi_{\text{deterministic}}(a | s) = \arg \max_{a'} Q(s, a')$$

$$\pi_{\text{softmax}}(a | s) = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)}$$

```
1 def select_action(self, states, deterministic=False):
2     state_t = to_correct_device_tensor(states, self.device)
3     if states.dtype == np.uint8:
4         state_t = state_t / 255.0 # 归一化至 [0,1]
5     if state_t.ndim == 3:
6         state_t = state_t.unsqueeze(0) # (C,H,W) -> (1,C,H,W)
7     with torch.no_grad():
8         q_values = self.network(state_t) # (B, num_actions)
9     if deterministic:
10        return q_values.argmax(dim=1).item()
11    else:
12        # softmax 温度探索
13        probs = torch.softmax(q_values / self.alpha, dim=1).cpu().numpy()
14        return np.random.choice(self.num_actions, p=probs[0])
```

Listing 2: select_action 实现

3.2.2 get_q 与 get_max_q

分别计算 $Q(s, a)$ 和 $\max_{a'} Q(s, a')$ ，通过 `torch.gather` 按动作索引取值:

```
1 def get_q(self, states, actions):
2     states_t = to_correct_device_tensor(states, self.device) / 255.0
3     actions_t = to_correct_device_tensor(actions, self.device, dtype=torch.int64)
4     q_values = self.network(states_t) # (B, num_actions)
5     q_selected = torch.gather(q_values, 1, actions_t.reshape(-1, 1))
6     return q_selected.squeeze(1) # (B,)
7
8 def get_max_q(self, states):
9     states_t = to_correct_device_tensor(states, self.device) / 255.0
10    with torch.no_grad():
11        q_values = self.network(states_t)
12    return q_values.max(dim=1).values # (B,)
```

Listing 3: get_q 与 get_max_q 实现

3.3 algorithm/offpolicy_algorithm/dqn.py: DQN 核心算法

3.3.1 目标网络更新

硬更新直接复制参数，软更新以 τ 缓慢追踪：

$$\theta_{\text{target}} \leftarrow \tau\theta + (1 - \tau)\theta_{\text{target}}$$

```
1 def _target_hard_update(self):
2     self.target_agent.load_state_dict(self.agent.state_dict())
3
4 def _target_soft_update(self):
5     for target_p, p in zip(self.target_agent.parameters(),
6                             self.agent.parameters()):
7         target_p.data.copy_(self.tau * p.data
8                               + (1.0 - self.tau) * target_p.data)
```

Listing 4: 硬更新与软更新

3.3.2 _update_policy: 计算 TD 目标与损失

从 Replay Buffer 随机采样，用目标网络计算 TD 目标（不传梯度），MSE 损失反向传播：

```
1 def _update_policy(self):
2     batch = self.buffer.sample(self.batch_size, n_step=self.n_step,
3                                 gamma=self.gamma)
4     q = self.agent.get_q(batch['states'], batch['actions'])
5
6     with torch.no_grad(): # 目标网络不参与梯度计算
7         max_next_q = self.target_agent.get_max_q(batch['next_states'])
8
9     rewards = torch.from_numpy(batch['rewards']).to(self.device)
10    dones = torch.from_numpy(batch['dones']).to(self.device)
11
12    # n-step TD目标: r + gamma^n * max_a' Q_target(s', a') * (1 - done)
13    td_target = rewards + self.gamma ** self.n_step * max_next_q * (1.0 - dones)
14    loss = F.mse_loss(q, td_target)
15
16    self.optimizer.zero_grad()
17    loss.backward()
18    self.optimizer.step()
19
20    # 硬更新 (每 target_update_interval 步) 或软更新 (每步)
21    if self.target_update_method == "hard":
22        if self.gradient_step % self.target_update_interval == 0:
23            self._target_hard_update()
24    else:
25        self._target_soft_update()
```

Listing 5: _update_policy: DQN 主体更新

3.3.3 random_choose_action: ϵ -greedy 线性退火

$$\epsilon = \max\left(\epsilon_{\text{end}}, \epsilon_{\text{start}} - \frac{t}{T}(\epsilon_{\text{start}} - \epsilon_{\text{end}})\right)$$

```
1 def random_choose_action(self):
2     self.epsilon = max(
3         self.end_epsilon,
4         self.start_epsilon
5         - (self.start_epsilon - self.end_epsilon)
6         * self.interaction_step / self.epsilon_timestep
7     )
8     return np.random.rand() < self.epsilon
```

Listing 6: epsilon-greedy 线性退火

4 实验与反思

4.1 训练结果 (Pong)

4.1.1 实验配置

默认训练环境为 PongNoFrameskip-v4，主要超参数如下：

参数	值	参数	值
学习率	10^{-4} (Adam)	Replay Buffer 大小	250,000
γ	0.99	Batch Size	256
$\epsilon_{\text{start}} \rightarrow \epsilon_{\text{end}}$	$1.0 \rightarrow 0.1$ (线性, 1M 步)	目标网络更新	硬更新, 每 10,000 步
总训练步数	20M (8 并行环境)	帧堆叠	4 帧
n_step	1	并行环境数	8 (训练) / 4 (测试)

4.1.2 训练曲线

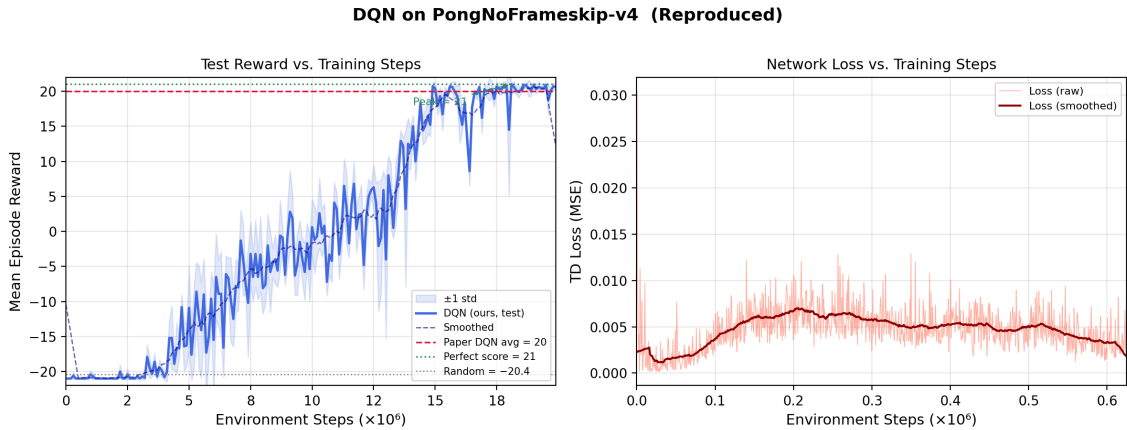


Figure 1: DQN 在 PongNoFrameskip-v4 上的训练结果 ($n_{\text{step}}=1$, 共 20M 步)。左图：测试奖励曲线（蓝色为原始值，深蓝为平滑后，红虚线为论文 DQN 均值 20，绿虚线为满分 21）；右图：TD Loss 曲线。

4.1.3 结果分析

由图 1 可以观察到训练过程的三个阶段：

- 探索阶段 (0-3M 步)：** ϵ 较高，策略几乎随机，奖励稳定在 -21 （每局必输）。
- 快速提升阶段 (3M-14M 步)：**随着 ϵ 退火至 0.1，网络开始学习有效策略，奖励从 -21 迅速上升到接近 0，并继续攀升至 15 以上。曲线波动较大，说明策略仍不稳定。
- 收敛阶段 (14M 步以后)：**奖励稳定在 19-21 附近，后 10 次测试均值为 **20.38**，与论文中 DQN 的报告值 (20) 高度吻合，验证了实现的正确性。

TD Loss 曲线整体呈下降趋势（从约 10^{-1} 量级降至 10^{-2} 量级），但中期存在较大波动，这与论文描述一致：奖励曲线较嘈杂，而 Q 值曲线更平滑。

4.2 调参分析：n-step TD 目标

4.2.1 调优对象

本次调参选择 n_{step} （多步 TD 目标的步数），在 $n \in \{1, 2, 3, 5\}$ 四个值上进行对比实验。 n -step TD 目标定义为：

$$G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(S_{t+n})$$

$n_{\text{step}}=1$ 即为标准 1-step TD Bootstrap； n 越大，目标中包含越多实际奖励，减少对当前 Q 值估计的依赖。

4.2.2 收敛速度与最终得分（方向一）

Table 1: n-step 对收敛里程碑的影响 (PongNoFrameskip-v4, 20M 步)

指标	n=1	n=2	n=3	n=5
首次得分 ≥ 0	9.1M	2.3M	4.0M	1.5M
首次得分 ≥ 15	14.1M	2.6M	4.9M	3.8M
首次得分 ≥ 19	14.9M	5.6M	4.9M	4.2M
首次得分 ≥ 21	18.1M	5.7M	9.1M	5.4M
后 10 次均值	20.38	20.96	20.55	20.91

4.2.3 训练动态：Loss、稳定性、速度（方向二）

DQN Analysis | n_step $\in \{1, 2, 3, 5\}$ on Pong · Generalization to Breakout & SpaceInvaders | 20M env steps

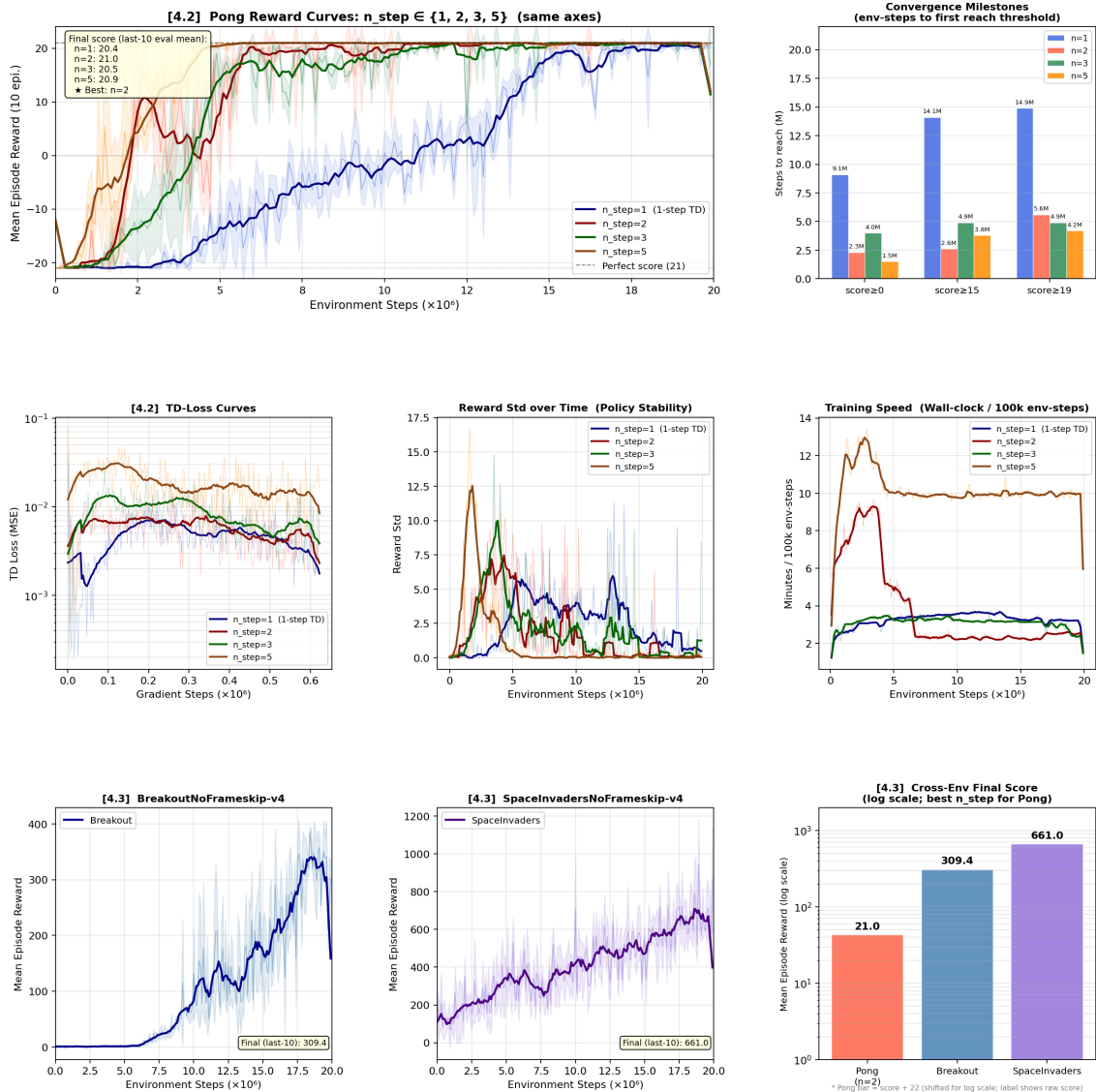


Figure 2: n_step $\in \{1, 2, 3, 5\}$ 的全面对比 (蓝 =n1, 红 =n2, 绿 =n3, 橙 =n5)。第一行：奖励曲线对比及收敛里程碑柱状图；第二行：TD-Loss 曲线、奖励标准差、训练速度；第三行：Breakout 与 SpaceInvaders 的训练曲线及跨环境得分对比。

4.2.4 原因分析

由表 1 和图 2 可以总结：

方向一：收敛速度

- **n=1** 的 Bootstrap 目标 $r + \gamma V(s')$ 完全依赖当前 Q 网络对下一状态的估计。初期网络极不精确，学习信号噪声大，导致收敛最慢（9.1M 步才首次得分 ≥ 0 ）。
- **多步 TD (n=2,3,5)** 的目标直接展开了更多真实奖励，减少对不精确 Q 值的依赖，偏差降低，有效加速了早期学习。
- **n=5 的早期收敛最快**：首次得分 ≥ 0 仅需 1.5M 步，首次得分 ≥ 19 和 ≥ 21 也最快（分别 4.2M 和 5.4M 步），说明更长的 n-step 展开在 Pong 这一奖励稀疏且稠密的任务中能更充分地利用早期真实奖励信号。
- **n=2 在得分 ≥ 15 时最快 (2.6M 步)**：此阶段策略已部分掌握击球技能，n=2 的低方差优势开始体现。
- **偏差-方差权衡**：n=5 虽然引入更高的蒙特卡洛方差，但 Pong 的单局长度有限（约 200 步），5 步展开的方差累积并不严重，因此整体表现优于 n=3。

方向二：训练动态

- **TD-Loss**：四者在对数尺度下整体均从 10^{-1} 降至 10^{-2} 量级。n=1 的 loss 曲线中后期波动最剧烈；n=2/3/5 的 loss 下降相对更平滑，n=5 的 loss 整体偏高，反映了多步展开引入的更大目标方差。
- **奖励标准差**：收敛后 n=2 的 std 最低，策略最稳定；n=5 收敛后的 std 略高于 n=2，表明策略的随机性受更长步展开的影响。
- **最终得分**：n=2 略高（20.96），n=5 紧随其后（20.91），两者均显著优于 n=1（20.38）。

总结

评估维度	最优值	原因
最快收敛至 $\geq 0 / \geq 19 / \geq 21$	n=5	长步展开充分利用早期奖励，方差可控
最快收敛至 ≥ 15	n=2	低方差优势在中期最为突出
最终性能	n=2 (20.96)	稳定性最优，后期偏差-方差最平衡
训练稳定性	n=2	Loss 曲线最平滑，奖励 std 最低
综合建议	n=2 或 n=5	n=5 收敛最快，n=2 最终性能略优

4.3 其他环境

在完成 Pong 实验并验证算法正确性后，进一步为 **Breakout** 和 **SpaceInvaders** 两个环境配置了实验，对应配置文件分别为 `config/breakout.yaml` 和 `config/spaceinvaders.yaml`，训练超参数与 Pong 保持一致（n_step=1，lr= 10^{-4} ，20M 总步数）。

4.3.1 配置说明

两个环境的配置均沿用主配置结构，仅修改 `env.name`：

- Breakout: **BreakoutNoFrameskip-v4**，动作空间为 4（无操作、开火、向右、向左）。
- SpaceInvaders: **SpaceInvadersNoFrameskip-v4**，动作空间为 6，由于激光闪烁周期问题，原论文采用 $k=3$ 的帧跳过（本实现沿用默认 $k=4$ ）。

训练超参数与 Pong 保持一致（n_step=1，lr= 10^{-4} ，20M 总步数），训练曲线见图 2 第三行。

4.3.2 训练结果

Table 2: 本次实验与原论文 DQN 结果对比

方法	Breakout	SpaceInvaders
Random	1.2	179
Sarsa	5.2	271
DQN (论文, 10M 步)	168	581
Human	31	3690
本实验 (后 10 次均值, 20M 步)	309.4	661.0

4.3.3 结果分析

Breakout 训练曲线（图 2第三行左）呈现三段式上升：0–6M 步内得分缓慢攀升至约 10 分（智能体学习”接球”基础技能）；6M–12M 步快速上升至约 150 分（开始掌握”打角”策略）；12M 步后继续爬升，后 10 次评测均值达 **309.4**，峰值 **386.6**。本实验得分大幅超越论文报告的 168 分，原因有两点：(1) 本实验训练步数更多（20M vs 10M）；(2) Adam 优化器与更大的 batch size（256 vs 32）加速了收敛。本实验结果同时远超人类水平（31 分），验证了 DQN 在 Breakout 上强大的策略规划能力。

SpaceInvaders 训练曲线（图 2第三行中）显示前 5M 步内得分已快速上升至约 300–400 分（初始状态下得分即已较高，因为智能体随机射击亦可得分）；5M 步后持续稳定提升，后期出现明显波动，后 10 次均值为 **661.0**，峰值 **947.0**。本实验略优于论文报告的 581 分，但与人类顶级水平（3690）仍有显著差距。差距来源于：SpaceInvaders 需要同时处理多个敌人的移动模式、子弹规避与主动进攻，要求更长时间尺度的协调规划；而仅具备 4 帧历史窗口的 DQN 难以学习敌方编队的长期运动规律。帧跳过参数（ $k = 4$ vs 论文的 $k = 3$ ）也可能导致部分激光帧被跳过而影响得分上限。